

SOAR Playbook Architecture: Phishing Response

Task 4 — SOAR Architecture Design | THE ARZENS Internship, AI, Automation & Security Engineering Track

This document specifies the architecture of the Phishing Response playbook implemented in Task 5 (phishing_playbook.py). It selects Phishing Response as the target use case because phishing is high-volume, has well-defined machine-checkable indicators (sender domain, embedded URLs, attachment hashes), and maps cleanly onto a tiered, partially-automated response model that keeps a human in the loop for anything destructive. The workflow diagram on the following page shows the full pipeline from trigger to audit log.

1. Trigger

The playbook is triggered by one of two events: (a) an end user reporting a suspicious email through a mail-client plugin, or (b) the corporate mail gateway automatically forwarding an email that matched a heuristic spam/phishing filter but was not confident enough to block outright. In both cases the trigger payload is a structured record: sender, subject, body, and any attachment_hash — matching the sample_email.json schema used in Task 5.

2. Conditions

Before any processing occurs, the playbook validates its input: the email record must contain a well-formed sender address and at least one of {body, URLs, attachment_hash} to analyze. Malformed or empty submissions are rejected and logged rather than silently ignored, so a broken upstream integration is visible instead of quietly dropping reports.

3. Actions

Once validated, the playbook performs, in order: (1) indicator extraction — parsing the sender domain, any URLs in the body, and the attachment hash; (2) reputation checking — cross-referencing each indicator against a local blacklist file and, where configured, the VirusTotal API; (3) risk scoring — a weighted 0-100 score built from how many indicators matched, on which reputation source, and whether the domain is newly registered or otherwise anomalous; and (4) a disposition decision based on that score.

4. Outputs

The playbook produces three outputs regardless of disposition: a structured JSON/log record of the decision and evidence (playbook.log), an email notification to the security team summarizing the case and the action taken, and — for High severity only — an attempted containment action (domain/hash block, email quarantine). No output silently disappears; even a Low-severity "log only" case is written to the audit trail.

5. Human-in-the-loop

Disposition is tiered by risk score: High (≥ 70) triggers an automatic block, but that block is reversible via --rollback within a configurable window and is flagged to an analyst for confirmation rather than closed silently. Medium (30-69) never auto-acts; it is placed in an analyst review queue. Low (< 30) is logged only. This mirrors the three-tier auto-close / analyst-review / escalate pattern used in Task 1, applied here to

containment actions instead of alert triage.

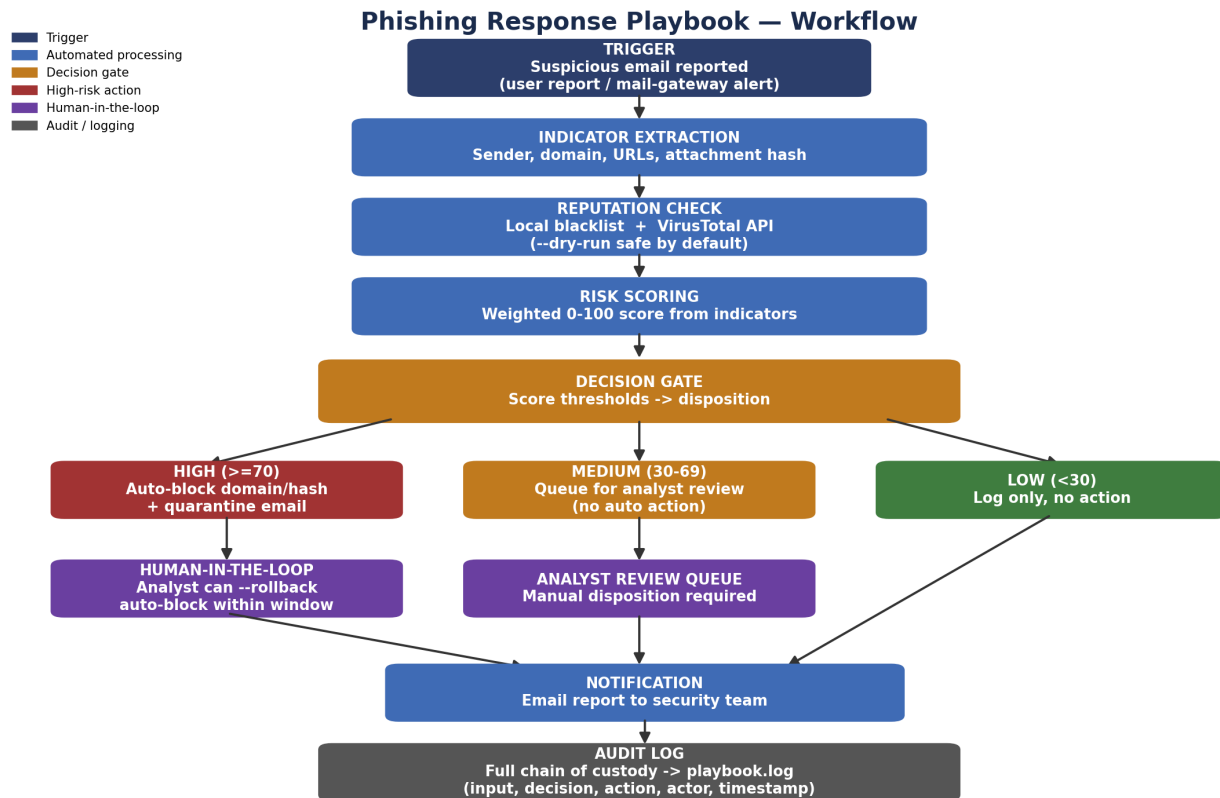


Figure 1 — Phishing Response playbook workflow, from trigger to audit log.

Safety Controls

Control	Description
Dry-run	The entire playbook can be run with <code>--dry-run</code> , which performs indicator extraction, reputation lookups, and scoring exactly as normal but logs the would-be action instead of executing it (no real block, no real email sent). This is the default mode for any new deployment or test.
Rollback	Every automated High-severity block is recorded with enough detail (indicator, timestamp, rule that triggered it) that <code>--rollback</code> can reverse it — removing the domain/hash from the active block list and logging the reversal with the analyst's identity and reason.
Rate limiting	Reputation lookups against the VirusTotal free tier are capped at 4 requests/minute (matching the documented free-tier limit) with local caching of recent lookups so repeated indicators don't re-spend the quota.
Input validation	All fields from the (untrusted) reported email are validated and size-limited before use — URLs and hashes are checked against expected formats via regex before being sent to any external API, to avoid malformed input reaching the reputation service or being logged in a way that breaks downstream parsers.

Audit Requirements

Every playbook run appends a structured entry to `playbook.log` capturing: the full chain of custody — who/what triggered the run, the extracted indicators, the reputation-check results and their source, the

computed risk score, the disposition reached, any action taken (or the fact that --dry-run suppressed it), and the timestamp of each step. Logs are retained for a minimum of 180 days in this reference implementation (configurable in config.yaml), which aligns with common SOC audit-retention practice for security-relevant events. Logs are append-only from the playbook's perspective (the tool never edits or deletes a prior entry) so the log itself can serve as evidence during an incident review or compliance audit.